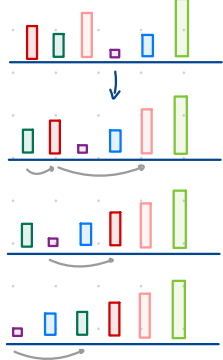


Bubble Sort



→ After i iterations the first or last i Elements are at their correct location

→ Max # Moves = $\frac{3n^2 - 3n}{2}$

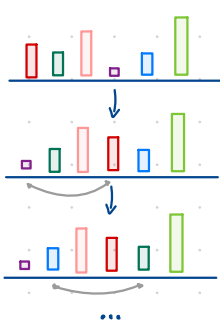
→ Runtime Complexity = $O(n^2)$

bubbleSort(A, n)

```
for i = 0 to n-1 do { n-1 }
  for j = 0 to n-i-2 do { (n-i-2) }
    if A[j] > A[j+1] do
      swap(A[j], A[j+1])
```

return A

Selection Sort



→ After i iterations the first or last i Elements are at their correct location

→ Max # Moves = $3(n-1)$

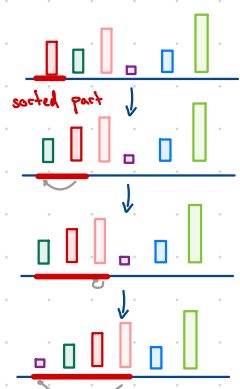
→ Runtime Complexity = $O(n^2)$

selectionSort(A, n)

```
for i = 1 to n-1 do { n }
  k = i
  for j = i+1 to n do { n-i-1 }
    if A[j] < A[k] do k = j
  swap(A[i], A[k])
```

return A

Insertion Sort



→ Sorted part grows with every iteration

→ Comparisons Max = $\frac{n^2 - n}{2}$

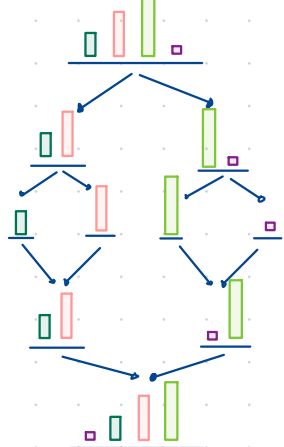
→ Moves Max = $\frac{n^2 - 3n - 4}{2}$

→ Runtime Complexity = $O(n^2)$

insertionSort(A, n)

```
for i = 2 to n do { n }
  j = i-1; t = A[i];
  while j > 1 & t < A[j] do { 2(n-1) }
    swap(A[j], A[j+1]); j--
  A[j+1] = t
return A
```

Merge Sort



Merge(A, l, r, m)

```
for i = l to m do B[i] = A[i]
for i = m+1 to r do B[m+i+1] = A[i]
```

A = 2 3 4 1 8 9 → B = 2 3 4 1 8 9

i = l, j = r

for k = i to r do

if B[i] < B[j] do A[k] = B[i], i++

else do A[k] = B[j], j++

$T(n) = \begin{cases} \Theta(1) & \text{if } n=1 \\ 2T(\frac{n}{2}) + \Theta(n) & \text{else} \end{cases}$

→ MM Case 2

$T(n) = \Theta(n^{\log_b a} \log_b n)$
 $= \Theta(n \log n)$

MergeSort(A, l, r)

if l < r then

m = floor((l+r)/2)

MergeSort(A, l, m)

MergeSort(A, m+1, r)

Merge(A, l, r, m)

Heap Sort

→ Properties of Heap:

1. Nearly complete Binary Tree
2. Each Node is greater or equal to all children

Heapify(A, i, n)

m = i; L = 2i; R = 2i+1;

if L ≤ n & A[L] > A[m] do m = L

if R ≤ n & A[R] > A[m] do m = R

if i ≠ m do

swap(A[i], A[m])

Heapify(A, m, n)

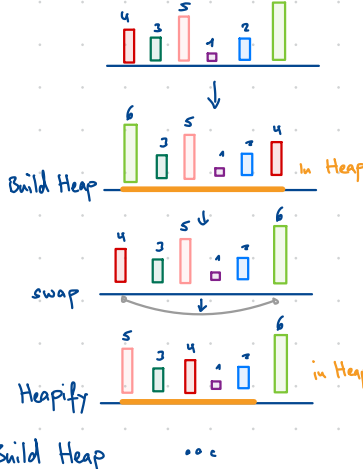
HeapSort(A, n)

s = n;

for i = floor(n/2) to 1 do Heapify(A, i, n) { Build Heap }

for i = n to 2 do

swap(A[i], A[1]); s--; Heapify(A, 1, s)



Quick Sort

HoarePartition(A, l, r)

x = A[l]; i = l-1; j = r+1;

while true do:

repeat j-- until A[j] ≤ x;

repeat i++ until A[i] ≥ x;

if i < j then swap(A[i], A[j])

else return i

QuickSort(A, l, r)

if l < r then

m = HoarePartition(A, l, r)

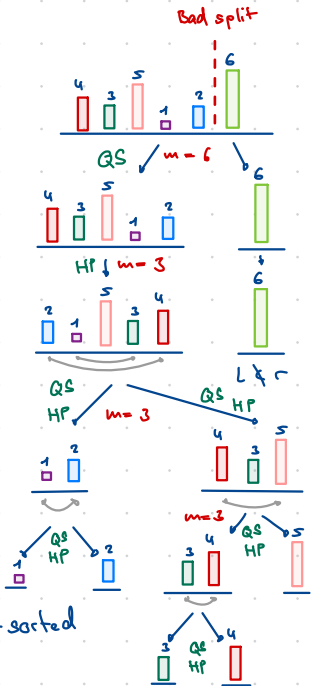
QuickSort(A, l, m-1)

QuickSort(A, m, r)

→ Worst Case = $O(n^2)$, if A is pre-sorted

→ Most often $O(n \log n)$

↳ Randomized x = rand A[l:r] → improvement



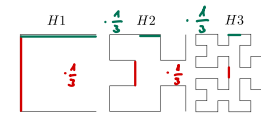
Recursion

→ Multiple Recursion: Procedure calls itself repeatedly

→ Mutual Recursion: Different Procedures call each other

→ Geometric Recursion: Hilbert Curve

↳ To "fill" a paper in one line



→ Recursive Problem = $\begin{cases} \text{Base Case} \\ \text{Recursive Call} \end{cases}$

Runtime Analysis

→ Insertion Sort:

Cost of Operation

Multiplier

$T(n) = c_1 n + (c_2 + c_3)(n-1) + c_4 \sum_{i=1}^n t_i + (c_5 + c_6) \sum_{i=2}^n (t_i - 1) + c_7(n-1) + c_8$

→ Best Case: Array is already sorted $t_i = 1$

$T(n) = c_1 n + (c_2 + c_3 + c_4 + c_7)(n-1) + c_8$
 $= an + b$

→ Worst Case: Array is sorted in reverse $t_i = i$

$T(n) = c_1 n + (c_2 + c_3 + c_7)(n-1) + c_4 \left(\frac{n(n+1)}{2} - 1 \right) + (c_5 + c_6) \frac{(n-1)n}{2} + c_8$
 $= an^2 + bn + c$

→ Mostly the worst case RT is in focus

→ Number of iteration on innermost loop is crucial

Correctness

Precondition

legal Input

Algorithm

Postcondition

desired Output

→ Invariants: Assertions that are true, every time they are reached

↳ Consists of three statements:

- ① Initialization: Inv. is true before first iteration
- ② Maintenance: true before iteration → true after iteration
- ③ Termination: when loop terminates → useful property is given

→ Example in Insertion Sort: $\forall j \in [1, i-1]: A[j] \leq A[j+1]$

① Init.: $i = 2 \rightarrow A[1 \dots 1]$ is trivially sorted

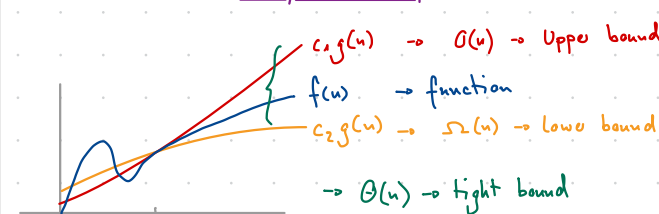
② Maint.: $A[1 \dots i-1]$ is sorted $\rightarrow A[i]$ is inserted into right place $\rightarrow A[1 \dots i+1]$ is sorted

③ Term.: $i = n+1 \rightarrow A[1 \dots n]$ is sorted

→ Special Case Analysis: Testin Algo with Edge cases

↳ for example: A = [5], A[k] = -1 ...

Asymptotic Complexity



→ Upper Bound: $f(n) \leq c_1 g(n) \quad \forall n \geq n_0$
 $c > 0, n_0 \geq 0$

Case 3

$\lim_{n \rightarrow \infty} \left(\frac{g(n)}{f(n)} \right) = +\infty$

→ Lower Bound: $f(n) \geq c_2 g(n) \quad \forall n \geq n_0$
 $c > 0, n_0 \geq 0$

Case 2

$\lim_{n \rightarrow \infty} \left(\frac{g(n)}{f(n)} \right) = c$

→ Tight Bound: $c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \forall n \geq n_0$
 $c_1 > 0, c_2 > 0, n_0 \geq 0$

Case 1

$\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = k, k \in \mathbb{R}$

Recurrences

→ Runningtimes of Algo's with recursive calls

- Three ways of solving:
- ① Master Method
 - ② Repeated Substitution
 - ③ Substitution
 - ④ Recursion Trees

→ Master Method: if $T(n) = a T(\frac{n}{b}) + f(n)$
 $a, b > 0 \quad \lim_{n \rightarrow \infty} f(n) > 0$

↳ $T(n)$ describes the Runtime of Divide & Conquer Algo
 Three Cases: compare $f(n)$ and $n^{\log_b a}$

① $f(n) = O(n^{\log_b a - \lambda}), \lambda > 0$

→ $f(n)$ grows slower than $n^{\log_b a}$

→ Work at leaves of Tree dominates

→ $T(n) = \Theta(n^{\log_b a})$

② $f(n) = \Theta(n^{\log_b a})$

→ $f(n)$ and $n^{\log_b a}$ are asymptotically identical

→ Work is evenly distributed

→ $T(n) = \Theta(n^{\log_b a} \log_b n)$

③ $f(n) = \Omega(n^{\log_b a + \lambda}), \lambda > 0$

→ $f(n)$ grows faster than $n^{\log_b a}$

→ Work at root dominates

→ $T(n) = \Theta(f(n))$

→ Repeated Substitution:

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + cn \\ &= 2\left(2T\left(\frac{n}{4}\right) + \frac{cn}{2}\right) + cn \\ &= 4T\left(\frac{n}{4}\right) + 2cn \\ &\dots \end{aligned}$$

Proceed until pattern gets obvious

$$T(n) = 2^i T\left(\frac{n}{2^i}\right) + icn \rightarrow \text{find value that makes sense}$$

$$T(n) = nT(1) + cn \log n \rightarrow \Theta(n \log n)$$

→ Substitution

→ Guess form of Solution
→ Proof by induction

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

→ Guess $T(n) = O(n^3)$ → show that $T(n) \leq cn^3$

$$T(n) \stackrel{IH}{\leq} 4c\left(\frac{n}{2}\right)^3 + n$$

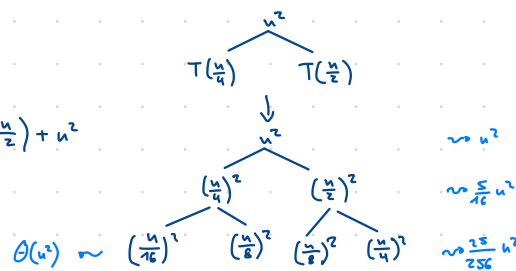
$$= c\frac{n^3}{2} + n$$

$$= cn^3 - (c\frac{n^3}{2} - 2)$$

$$T(n) < cn^3 \rightarrow T(n) = O(n^3) \rightarrow \text{try with smaller bounds (not that tight)}$$

→ Recursion Tree

$$T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{n}{4}\right) + n^2$$



Dynamic Data Structures

→ Linked Lists: root → A → B → C → ...
* p → to follow p
Ap → address of p

→ Queue's: FiFo root → B → C → D → ...
insertion = O(1)
deletion = O(n)

→ Stack's: LiFo root → B → C → D → ...
insertion = O(1)
deletion = O(1)

→ Stack's and Queue's can also be implemented with Arrays

Stack: $t = \text{tail}$
 $h = \text{head}$
Push(x): $s[t] = x; t++;$
Pop(): $t--;$ return $s[t]$

Queue: $t = \text{tail}$
 $h = \text{head}$
Enqueue(x): $Q[t] = x; t = (t+1) \bmod n;$
Dequeue(): $i = h; h = (h+1) \bmod n;$ return $Q[i]$

→ Some Operations need to change the root of an ADT

→ This can lead to problems since functions don't affect their arguments globally. Three solutions:

① Use a Sentinel: Dummy elmt like second root } kind of

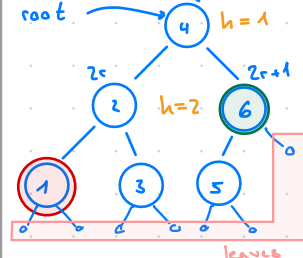
② Feed Function with a pointer to pointer type } the same

③ Let Function return elmt and reassign root to that

Trees

→ Binary Tree: each Node has max two children

→ Binary Search Tree: $\left\{ \begin{array}{l} \text{Node} \rightarrow \text{left} \leq \text{Node} \\ \text{Node} \rightarrow \text{right} \geq \text{Node} \end{array} \right\}$ Searchtime $\Theta(\lg n)$



→ Smallest elmt: furthest left

→ Largest elmt: furthest right

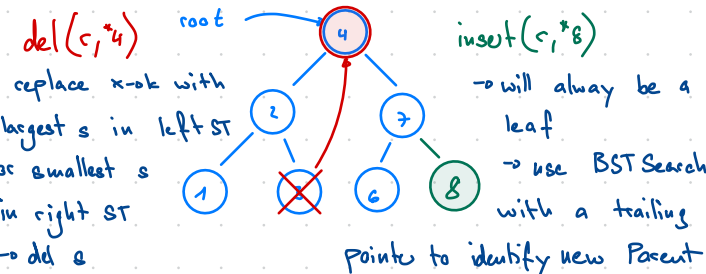
find Min/Max(r)
While $r \rightarrow \text{left/right} \neq \text{NIL}$
 $r = r \rightarrow \text{left/right}$
return r

inOrderTreeWalk(r)
if $r \neq \text{NIL}$ do
preOrder($r \rightarrow \text{left}$)
visit(r)
postOrder($r \rightarrow \text{right}$)

BSTSearch(r, x)
if $r = \text{NIL} \vee r \rightarrow k = x$
return r
if $r \rightarrow k < x$
return BSTSearch($r \rightarrow \text{right}, x$)
return BSTSearch($r \rightarrow \text{left}, x$)

→ Successor: smallest elmt larger than x

→ Insertion & Deletion: use trailing pointers



Red and Black Trees

→ to keep BST's balanced Properties:

→ Searchtime $O(\lg n)$

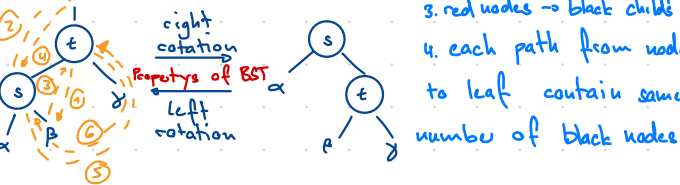
→ Operations take longer $O(\text{height})$

1. root is black

2. every leaf is black

3. red nodes → black child

4. each path from node to leaf contain same number of black nodes



Following Properties:

$bh \geq \frac{1}{2}h$

$2^{\frac{h}{2}} \leq 2^h \leq n+1$

$h \leq 2 \log_2(n+1)$

→ Therefore the height of an RBT is bound by $2 \log_2(n+1)$

RBTInsert(x)

$s = \text{NIL}; \rightarrow \text{delayed pointer}$

$x = \text{root};$

While $x \neq \text{NIL}$ do

$s = x;$

if $t \rightarrow k < x \rightarrow k \{x = x \rightarrow \text{left}\}$

else $\{x = x \rightarrow \text{right}\}$

t ← parent = s; t ← col = red;

if s = NIL {root = t}

else { if t → k < s → k { s → left = t }

else { s → right = t }

insertFixup(t);

→ There are four Cases (and they're Mirror Cases) for insertion

Assumption: p is left child of g.

Base Case: parent is black

→ No property is violated

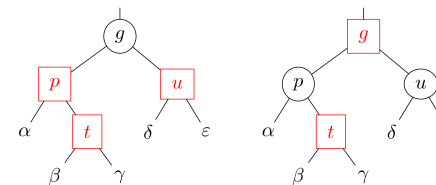
t = new Node

p = parent

g = grandparent

n = uncle

Case 1: parent and uncle are red



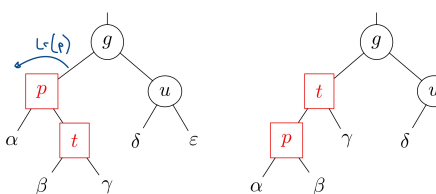
p → col = u → col = black

g → col = red

set t = g

→ propagate upwards from g (max lg(n) times)

Case 2: p is red, n is black, t is right child



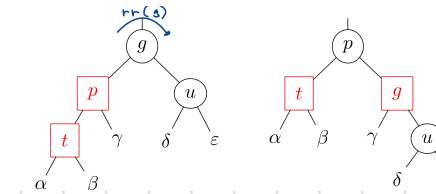
leftrotate(p)

set t = p

→ move downward once

→ Case 2 always leads to Case 3

Case 3: p is red, n is black, t is left child



p → col = black

g → col = red

rightrotate(s)

→ don't move

→ Mirror Cases appear if p is a right child

→ swap L & R in every action...

→ Deletion: five Cases (and they're MC)

→ first apply simple BSTdeletion(s)

→ Replace s by largest child of left tree x

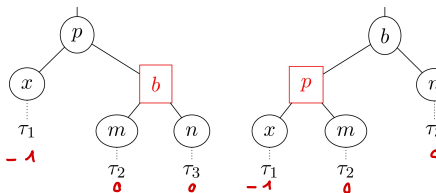
and delete x

Assumption: s was left child

Base Case: s was red

→ No properties violated

Case 1: s was black, b is red



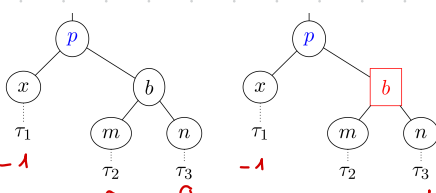
p → col = red

b → col = black

leftrotate(p)

Leads to case 2, 3 or 4

Case 2: s was black, b is black, b → left is black, b → right is black



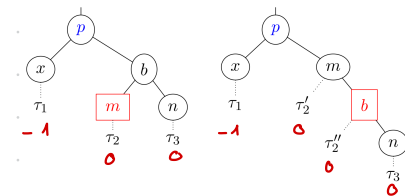
b → col = red

set x = p

→ black height of both sides is reduced

→ propagate upwards

Case 3: s was black, b is black, b → left = red, b → right = black



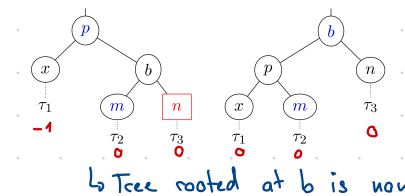
→ b → left → col = black

→ b → col = red

→ right rotate(b)

→ Case 4 is reached

Case 4: s was black, b is black, b → right is red



→ b → col = p → col

→ p → col = black

→ b → right → col = black

→ leftrotate(p)

→ To accomplish properties after deletion one does basically either:

① restructure → solves problem locally

② recolor → propagates problem upwards

→ Since C4 is the only one leaving a balanced tree, is it always the last operation??

Hash Tables

→ With Hashtable we compromise Time with space

→ Hashfunctions are used to compute a slot using the key

→ Item with key k hashes to slot h(k)

→ Hashfunctions are often designed using:

→ Division Method: $h(k) = (k \bmod m) + b, m \in \text{Primes}$

→ Multiplication Method: $h(k) = \lfloor m(kA \bmod 1) \rfloor + 1, 0 < A < 1$

→ If two values hash to same slot we can solve using:

① chaining: Items get put in linked list

→ Slot contains either NIL or pointer to linked list

→ Load Factor: average length of LL $\alpha = \frac{\# \text{ items}}{\# \text{ slots}}$

→ Search take $O(1+\alpha)$ time

② Linear Probing: Multiple positions are tried, linear stepsize

→ $h(k, i) = (h(k) + c_1 i) \bmod m$

→ stepsize should ensure that all slots are tried (often $c_1 = 1$)

→ table can not be overfilled → max $\alpha = 1$

③ Quadratic Probing: Multiple positions are tried, quadratic stepsize

→ $h(k, i) = (h(k) + c_1 i + c_2 i^2) \bmod m$

→ faster if HT is big and relatively full

④ Double Hashing: Stepsize is defined by separate hashfunction

→ $h(k, i) = h_1(k) + i \cdot h_2(k) \bmod m, c_1 = 0$

→ $h_2(k)$ must be relatively prime to table size m

→ Cost of search: $\frac{1}{(1-\alpha)}$ → Unsuccessful = $\frac{\ln(1/(1-\alpha))}{\alpha}$

→ Deletion in HT is dangerous, use special value for deletion

Dynamic Programming

- So far, in Divide and Conquer Algos, we often solved the most basic step multiple times, this can be expensive
- Target with DP is to save the results during the run of Algo
- For example we can Towers of Hanoi recursively and then improve

→ The Knapsack problem tries to find the maximum value to be carried in a sack with certain weight-capacity

```
KnapsackRecursive(W, w[3], v[3], u)
if w==0 || u==0 {return 0}
if w[u-1] > W {
    return KnapsackRecursive(W, w, v, u-1)
} else {
    return max(
        v[u-1] + KSR(W-w[u-1], w, v, u-1),
        KSR(W, w, v, u-1)
    )
}
```

$O(2^n)$

→ We try to save the solutions of our subproblems

```
KnapsackDP(W, w[3], v[3], u)
dp = Array [u+1][W+1]
for i = 0 to u {
    for w = 0 to W {
        if i==0 || w==0 {dp[i][w]=0}
        else if w[i-1] ≤ W {
            dp[i][w] = max(
                v[i-1] + dp[i-1][w-w[i-1]],
                dp[i-1][w]
            )
        } else {
            dp[i][w] = dp[i-1][w]
        }
    }
}
return max(dp[u][W])
```

$O(nw)$

→ A matrix is initialized where we try to find the max value for every weight

	0	1	2	...	W
0	-	-	-	-	-
1	-	-	2	-	2
2	-	-	-	...	4
...	-	-	-	-	...
n	-	-	-	-	-

Graphs

→ Graphs consist of Set of vertices and Set of edges $E \subseteq V \times V$

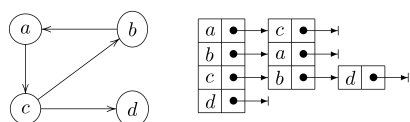
→ Every vertex has an in- and outdegree

→ Some more definitions:

- Connected Component: maximum Subgraph which is connected
- Tree: connected graph with n vertices and $(n-1)$ edges

Every vertex is connected to every other vertex

→ Adjacency List:
Space Required: $O(|V| + |E|)$



→ Adjacency Matrix:
Space Required: $O(|V|^2)$

	a	b	c	d
a	0	0	1	0
b	1	0	0	0
c	0	1	0	1
d	0	0	0	0

→ Nodes of a graph can have multiple attributes, one often used is "Color" to indicate status in traversal of graph

White: Undiscovered Gray: discovered but not all adj. vertices explored Black: discovered and all adj. vertices explored

Breadth first Search:

→ examine the distance to starting vertex outwards

```
BFS(G, s)
foreach v ∈ G.V do
    v.col = white; v.dist = ∞; v.pred = NIL;
s.col = gray; s.dist = 0;
InitializeQueue(q); enqueue(s)
While Q is not empty {
    v = dequeue();
    for each u ∈ v.adj {
        if u.col == white {
            u.col = gray; u.dist = v.dist + 1;
            u.pred = v; enqueue(u);
        }
    }
    v.col = black;
}
```

→ travel outwards from starting vertex

→ Computed distances are always the same but the BFS Trees can vary depending on order of visits

$RT = O(|V| + |E|)$

→ covers all vertices reachable from start

Depth First Search:

→ Two Algo's: First searches one Tree, second calls first until entire graph is covered

```
DFS Tree(G, s)
s.col = gray; time++; s.starttime = time;
foreach u ∈ s.adj {
    if u.col == white {
        u.pred = s; DFS Tree(G, u);
    }
}
s.col = black; time++; s.endtime = time;
```

→ travels downwards

→ each DFS Tree call adds new Tree to forest

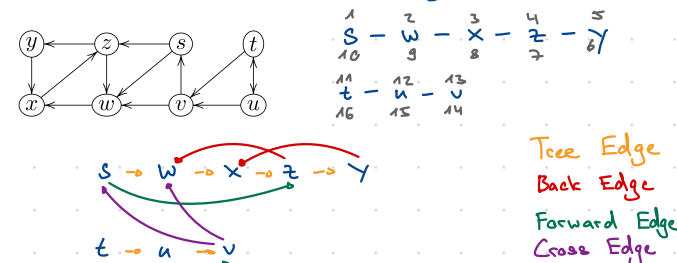
```
DFS Graph(G)
foreach v ∈ G.V {
    v.col = white; v.pred = NIL;
}
time = 0; (global)
foreach v ∈ G.V {
    if v.col == white { DFS Tree(G, v); }
}
```

→ every vertex is visited

$RT = O(|V| + |E|)$

→ DFS Edge Types:

- Tree Edge: Gray - White
- Back Edge: Gray - Gray
- Forward Edge: Gray - Black → From Ancestor to Descendant
- Cross Edge: Gray - Black → Edges between Trees



→ Behaviour difference if one Tree or entire Forest is searched between DFS & BFS can be changed in function call

→ Directed Acyclic Graphs (DAG's) are directed graphs which don't contain any cycles

→ Graph G is acyclic iff $DFS(G)$ has no Back Edges

```
DAGcheck(v)
v.col = gray;
for u ∈ v.adj {
    if (u.col == white) { DAGcheck(u); }
    else if (u.col == gray) { is DAG = false; }
}
v.col = black;
```

```
global is DAG = true
for v ∈ G.V { v.col = white; }
for v ∈ G.V { if v.col == white { DAGcheck(v); } }
```

→ Topological Sorting is used to sort DAG's
↳ is built on DFS

```
Topo Sort(G)
DFS Graph(G)
→ edit DFS Tree so that if the endtime is set, vertex v is inserted at beginning of a Linked List
return Linked List
```

$O(|V| + |E|)$

Minimum Spanning Trees

→ Weighted Graphs: each edge has a weight

→ MST is subgraph of G that visits all v with minimum cost → always acyclic

→ Prim Algo determines MST

```
Prim(G, w[3], s)
foreach v ∈ G.V { v.key = ∞; v.pred = NIL; }
s.key = 0; initialize_Min_PQ(G);
While Min_PQ is not empty {
    v = extract_Min(PQ);
    foreach u ∈ v.adj {
        if u ∈ PQ & w[u,v] < u.key {
            u.key = w[u,v];
            u.pred = v;
            DecreaseKey(PQ, u, w[u,v]);
        }
    }
}
```

$PQ = \text{Array: } O(|V|^2)$
 $PQ = \text{Heap: } O(E \log V)$

↳ Can be more efficient if heap is used

→ The DecreaseKey() call can vary dependent on Data Structure used as PQ
↳ if Heap is used → heapify()

Single-Source Shortest Path

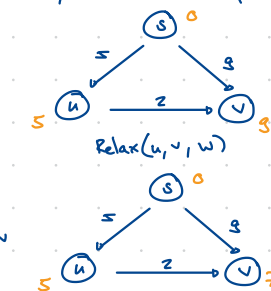
→ Shortest Path from source vertex to any other

→ All subpaths are shortest paths

→ SP's do not contain cycles → Negative Edge Cycles explode the Bus

→ Relaxing an edge (u,v) means looking for ways to reach v faster

```
Relax(u, v, w[3])
if v.dist > u.dist + w[u,v] {
    v.pred = u;
}
```



→ Bellman Ford algo relaxes all $v \in G.V$

→ If negative weight cycle is detected $BFC()$ return false

```
BellmanFord(G, w[3], s)
foreach v ∈ G.V {
    v.dist = ∞; v.pred = NIL;
}
for i = 1 to |G.V|-1 {
    foreach (u,v) ∈ G.E { Relax(u, v, w); }
}
foreach (u,v) ∈ G.E {
    if v.dist > u.dist + w[u,v] {
        return false;
    }
}
return true;
```

$O(|E| \cdot |V|)$

→ Finding shortest Paths in DAG's is much easier
↳ Dijkstra's Algo is $O(|V| + |E|)$

```
Dijkstra(G, w[3], s)
foreach v ∈ G.V {
    v.dist = ∞; v.pred = NIL;
}
s.dist = 0; initialize_Min_PQ(G.V);
while PQ is not empty {
    v = extract_Min(PQ);
    foreach u ∈ v.adj {
        Relax(u, v, w);
        DecreaseKey(PQ, u, u.dist);
    }
}
```

→ $|V|$ times faster than Bellman Ford

$O(|V| + |E|)$

→ Dijkstra can't handle negative weights
↳ use Bellman Ford instead

